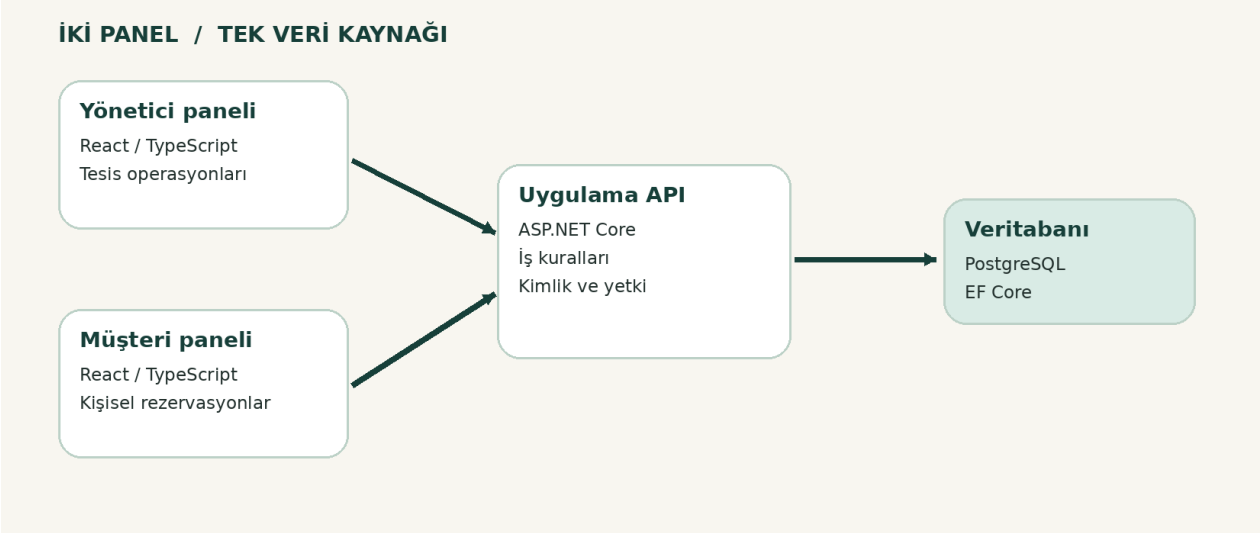
ALPINEOS / TEKNIK INCELEME

Teknik Mimari ve Veri Modeli

Mühendislik kararlarım, temel sınıflar ve iş kuralları

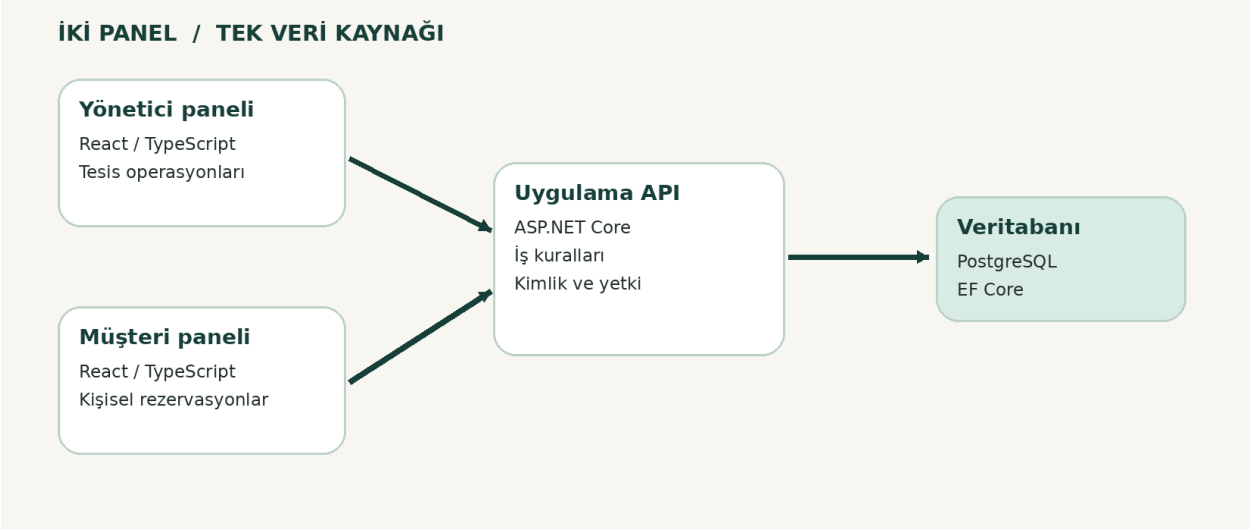


Hazırlayan / Fatih Yılmaz

Üniversite projesinin yeniden tasarlanmış uygulama çalışması

1 / Genel yaklaşım

Bu projeyi küçük parçalara ayrılmış ancak tek bir backend servisiyle çalışan modüler bir yapı olarak tasarladım. Yönetici ve müşteri panelleri farklı ekranlara sahip olmasına rağmen aynı API'ye konuşuyor. Böylece kiralama, konaklama ve ödeme kayıtlarının iki tarafta farklılaşmasını önlemeyi amaçladım.



Şekil 1. Uygulamanın çalışma yapısı.

Katman	Kullandığım teknoloji	Sorumluluk
Arayüz	React / TypeScript / Vite	Sayfalar, formlar, kullanıcı etkileşimi
Sunum	Nginx	Arayüzün sunulması ve API yönlendirmesi
Backend	C# / ASP.NET Core Minimal API	İş kuralları, erişim ve doğrulama
Veri erişimi	Entity Framework Core / Npgsql	Varlıklar ve PostgreSQL işlemleri
Kalıcı veriler	PostgreSQL	İlişkiler, kısıtlar, tutarlılık
Çalıştırma	Docker Compose	Web, API ve veritabanı servisleri

2 / Kodun organizasyonu

Dosya / klasör	Ne için kullanılıyor?
api/Program.cs	Servis başlangıcı, middleware, kayıtlar
api/Models/Entities.cs	Veri varlıkları ve ilişkiler
api/Data/ResortDb.cs	DbContext ve veri şeması
api/Features/	Kiralama, konaklama, etkinlik ve diğer API işlevleri
api/Security/AuthEndpoints.cs	Personel ve müşteri kimlik doğrulaması
api/Services/Rules.cs	Ücret ve tarih aralığı gibi kurallar
web/src/App.tsx	Yönetici arayüzü
web/src/CustomerPortal.tsx	Müşteri arayüzü
web/src/api.ts	API isteği ve oturum etkileşimi
compose.yaml	Servislerin birlikte çalıştırılması

3 / Veri modelini nasıl ayırdım?



Şekil 2. Başlıca iş alanlarının özet ilişkisi.

Eski projede kiralama, konaklama ve ödeme süreçlerinin birbiriyle daha sıkı bağlandığı alanları yeniden ayırdım. Bir müşteri birden çok kiralama ve konaklama kaydına sahip olabilir. Her kiralama belirli bir ekipmana; her konaklama belirli bir odaya bağlı. Ders veya etkinlik katılımları ise ayrı bir kayıt türü.

Varlık	Önemli alan / ilişki
Customer	Müşteri profili, oturum ve kişisel işlemler
Equipment / Maintenance	Demirbaş kimliği, fiyat ve bakım geçmişi
Rental	Müşteri + ekipman + işlem aralığı + durum + toplam
Room / Stay	Oda + kapasite + konaklama kaydı + durum
Activity / Enrollment	Etkinlik bilgisi, kontenjan, katılım
Department / Employee	Çalışanın organizasyon içindeki yeri
Payment / AuditEvent	Finans hareketi ve işlem izi

4 / Rezervasyon çakışma kuralı

Bir kaynak için rezervasyon aralıklarını başlangıç dahil, bitiş hariç kabul ediyorum. Bu sayede bir rezervasyonun bittiği anda yeni bir rezervasyon başlayabiliyor. Kiralama ve konaklama işlemlerinde API önce uygunluğu kontrol ediyor, fakat eşzamanlı istekleri güvenle ele alabilmek için PostgreSQL seviyesinde de kısıt uygulanıyor.

Önemli ilkem: "Müsaitlik kontrolü ekranda gösterilebilir; kesin veri bütünlüğü veritabanında sağlanmalıdır."



Şekil 3. Rezervasyonun doğrulama aşamaları.

5 / Ücret ve durum kuralları

- Ekipman kiralama: günlük ücret, yukarı yuvarlanan kullanım günü sayısıyla çarpılır.
- Konaklama: gecelik ücret, gece sayısı ile çarpılır.
- Toplam bedel, istemciden gelen değere güvenilmeden API içinde hesaplanır.
- Kiralama: Reserved → CheckedOut → Returned; alternatif olarak Reserved → Cancelled.
- Konaklama: Reserved → CheckedIn → Completed; alternatif olarak Reserved → Cancelled.
- Etkinlikler için kontenjan ve tekrar kayıt kontrolü yapılır.

6 / İzinler ve veri sahipliği

Kullanıcı türü	Örnek yetkiler
Admin	İşletme modülleri, hesaplar, denetim
Reception	Müşteri, kiralama, oda ve etkinlik akışları
Equipment	Ekipman ve bakım işlemleri
Slope	Pist durumunu değiştirme
Cashier	Manuel finans kaydı
Customer	Yalnızca kendi profili ve rezervasyonları

Özellikle müşteri tarafından yapılan isteklerde müşteri kimliğini formdaki bir alandan almak yerine oturum bilgisinden belirledim. Bu, başka bir müşterinin rezervasyonunun istenmesi gibi durumlara karşı temel koruma sağlıyor.

7 / Teknik borçlar

Mevcut uygulamanın tek tesisli veri modeli, sınırlı test kanıtı, gerçek ödeme entegrasyonunun bulunmaması ve sürümlü migration eksikliği geliştirmeye açık alanlar. Yeni özellik eklemeden önce bu temel noktaların iyileştirilmesini önceliklendiririm.